Applying Computational Science Final Project

The Palusznium Rush

Optimal mineral recovery using a Genetic Algorithm approach

Separation technologies are widely used to improve the purity of products. These technologies usually take the form of identical or near identical **separation units** (referred to as **units** for brevity) that are arranged in **circuits**. In minerals processing, for instance, the separation units are things like flotation cells or spirals, while in the upgrading of nuclear material the separator unit will be a centrifuge. Although the physical separation units are different, their key shared feature is that they will recover a proportion of the "valuable" material and will simultaneously recover a proportion of the "waste" material (Fig. 1). In general, it is hoped that the proportion of valuable material will increase in the separated output compared to the input mixture.

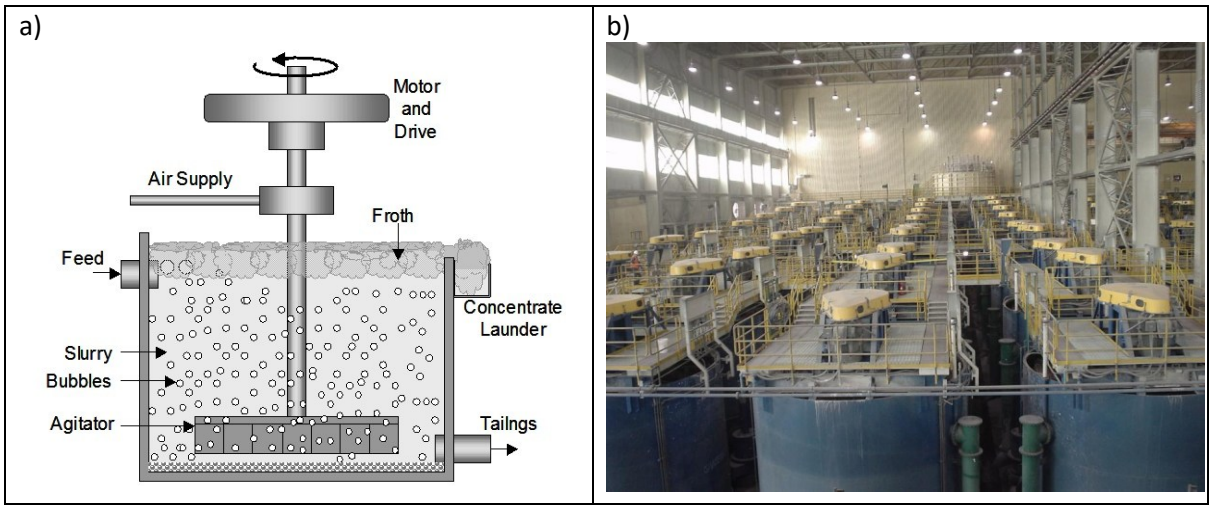


Figure 1: a) Schematic of a froth flotation cell (an example of a separation unit), which produces a concentrate stream via the froth with the rest of the material flowing out of the unit as tailings b) Picture of a large number of flotation cells arranged as a circuit

While a single separation unit in isolation will only recover a small proportion of the valuable material, multiple units can be combined together into circuits that can enhance recovery of the valuable material. The basic challenge is to design the circuit for optimal recovery. However, different circuit designs will result in different amounts of waste being recovered and thus different purities of the final product. While some circuit designs will be unambiguously better than others (i.e., produce both a better recovery and higher purity), for many designs there will be a compromise between the overall recovery (total mass of valuable material recovered) and purity (proportion of valuable material to the total material recovered). In such circumstances the optimum circuit will be an economic decision, based on the balance between how much you are paid for the product and how much you are penalised for a lack of purity.

In this project we are going to restrict ourselves to an overall circuit that produces three final products ("**streams"**), namely two **concentrate streams** of different materials and a **tailings stream**, and to units which produce two or three output streams, one or two concentrate streams and a tailings stream. The success of the circuit will be measured based on the final properties of the first two circuit products, the two concentrate streams, with a price paid per kg of the relevant primary valuable material and a penalty charged per kg of waste material or of the other valuable material in

the concentrate stream. The last product, the tailings stream, will be dominated by waste material and will be discarded at no cost.

These circuits can have simple rows of units with the tailings or concentrate being passed to the next unit along (but not both). The circuits can also involve recycles, where a stream is passed back to a unit nearer the beginning of the circuit (but not back to the same unit). This means that the number of potential circuits increases factorially with the number of units in the circuits. A brute force approach to find the optimal circuit is thus only feasible for circuits consisting of relatively few units (by the time there are 60 units in the circuit there are more potential circuit configurations than there are atoms in the universe!).

The large number of possible circuit configurations necessitates an optimisation algorithm to search for a solution. As the configurations are discrete and thus non-differentiable, standard gradient search algorithms won't work. There are a number of potential algorithms that can be applied to such problems and the one that we will be using is a **genetic algorithm**. The valuable materials that you are trying to recover in this project are "**palusznium" and "gormanium"**.

Methodology

Genetic Algorithms

Genetic algorithms, as their name implies, work in a manner not dissimilar to how natural selection works. The heart of the algorithm is a representation of the problem as a vector of numbers (the "genetic code" of the problem). In this problem, the genetic code represents the connections in the circuit (see Fig. 2). To start, a large number of these vectors need to be randomly generated and evaluated, with a single number assigned to represent the success of each vector (i.e., the performance of the circuit). The function that takes in the vector and returns a single performance number is often referred to as the **fitness function**. The convergence of the algorithm will usually be enhanced if it can be ensured that every one of the initial vectors is both unique and valid, though this may in itself be a computationally intensive task.

The set of random initial vectors will form the **parents** for the next generation of offspring vectors via a combination of two processes:

***Mutations* –** Random changes in the numbers in the parent vector.

***Crossover* –** This is roughly equivalent to sexual reproduction. In this process a portion of one parent vector is swapped with a portion of another parent vector. The motivation for swapping a portion of a parent vector with another rather than swapping individual values randomly is that, over successive generations, values that work well together will end up next to one another in the vector (roughly equivalent to genes); preserving these portions of the genetic code is beneficial. In this problem, for instance, where the values in the vector represent connections in the circuit, a certain set of connections between a few units may be useful in more than one location in the overall circuit.

The steps in a basic genetic algorithm are as follows:

1) Start with the vectors representing the initial random collection of valid circuits.
2) Calculate the fitness value for each of these vectors.

You now wish to create *n* child vectors

3) Take the best vector (the one with the highest fitness value) into the child list unchanged (you want to keep the best solution).
4) Select a pair of the parent vectors with a probability that depends on the fitness value. In this case you might want to start by using a probability that varies linearly between the minimum and maximum finesses of the current population. This should be done "with replacement," which means that parents should be able to be selected more than once.
5) Randomly decide if the parents should crossover. If they don't cross, they both go to the next step unchanged. If they are to cross, a random point in the vector is chosen and all of the values before that point are swapped with the corresponding points in the other vector.
6) Go over each of the numbers in both the vectors and decide whether to mutate them (this should be quite a small probability). If the value is to be mutated, you should move the value by a random amount (you can decide the step size). In these circuits you can avoid clustering of the results near the minimum and maximum unit numbers by "wrapping" the change (i.e., don't artificially restrict the movement, rather use a modulus to bring it back within range). In the circuit problem values are essentially completely independent of one another as there is no reason to think that a connection to a unit with a number close to that of the currently

connected unit will be better than the connection to any other unit. This means that the potential step size can be set to be the same as the size of the valid range, though in other optimisation problems it may be useful to preferentially search values close to the current one.

7) Check that each of these potential new vectors are valid and, if they are, add them to the list of child vectors.
8) Repeat this process from step 4 until there are *n* child vectors
9) Replace the parent vectors with these child vectors and repeat the process from step 2 until either a set number of iterations have been completed or a threshold has been met (e.g., the best vector has not changed for a sufficiently large number of iterations).

Tuning the algorithm – You may notice that there are a number of hyper-parameters that you can tune when running these algorithms. These include:

- the number of offspring *n* that are evaluated in each generation;
- the probability of crossing selected parents rather than passing them into the mutation step unchanged (a recommended range is between 0.8 and 1);
- the rate at which mutations are introduced (recommended probabilities of 1% or lower)

You will need to investigate how these hyper-parameters change the rate of convergence achieved. The optimum parameters will depend on both the type of problem being investigated and the size of the problem (they will be different between your test problem used to develop the genetic algorithm and the actual circuit simulation, and they will vary with the number of units in the circuit).

Before the optimum circuit can be determined, though, we need to be able to evaluate the performance of a given circuit.

Modelling a Circuit

There are a few aspects that need to be covered in terms of calculating circuit performance. The first is representing the circuit as a vector:

Representing the Circuit – In these circuits the separation units can take in as many input streams as desired, but they must have a fixed number of output streams each:

- For **type A** units, there are **two** output steam, with one output stream, the “concentrate” containing significantly more of the valuable materials than the waste; and the other output stream the “tailings” containing more of the waste than valuable materials.
- For **type B** units, there are **three** output streams, two concentrate streams, each containing a higher percentage of one of the valuable materials, and a tailings stream containing more of the waste.

We can therefore represent the circuit in terms of two subvectors of integers

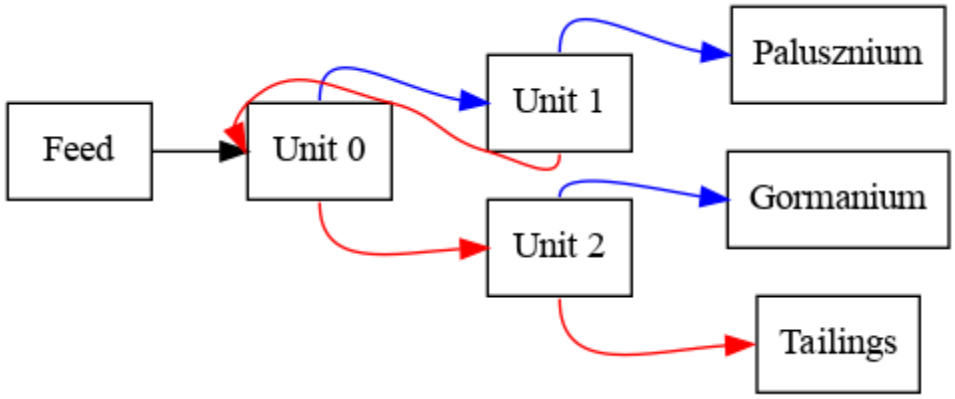
- A vector containing the size of the circuit (inputs, units and products) and the number of outputs of each unit.
- A vector containing the number of stream destinations, with each unit represented by two or three numbers, depending on the first vector.
    a. The first number is the unit number of the destination of the first high grade stream

b. The second number is either the unit number of the second high grade stream or the tailing
c. The third number (if present) is the unit number of the destination of the tailings.

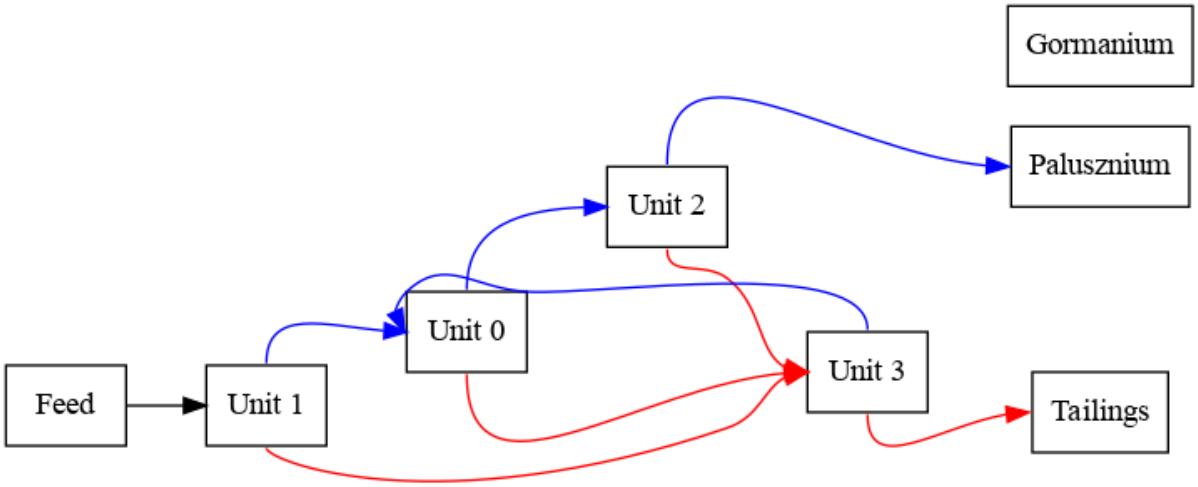
In addition to the *n* units, the streams can also be directed to the final tailings stream or either of final concentrate streams, which are the economic products to be evaluated.

As well as recording the destination unit of each output stream, the genetic code for the circuit should also include the destination unit of the circuit **feed** or input (to allow it to be changed). This means that if there are *n* units in the circuit, it will be represented by vector between 3*n*+1 and 4n+1 in length with the first *n* values being 2 or 3 and each subsequent value being in the range of zero to less than *n*+3 (0 to n-1 being the identifiers for the destination units, while a value of *n* represents the destination of the first (palusznium) product concentrate, n+1 the second (gormanium) product concentrate and *n+2* the destination of the final tailings), with the exception of the input feed's value, which should be between zero and *n-1* (the input feed should not be fed directly into one of the final product streams).

The following are a couple of examples of circuit vectors and the corresponding circuit layouts:



inputs	Units	Product Streams	Unit 0 outputs	Unit 1 outputs	Unit 2 outputs	Feed	Unit 0		Unit 1		Unit 2	
1	3	2	2	2	2	0	1	2	3	0	4	5



inputs	Units	Product Streams	Unit 0 outputs	Unit 1 outputs	Unit 2 outputs	Unit 3 outputs	Feed	Unit 0		Unit 1		Unit 2		Unit 3	
1	4	2	2	2	2	2	1	2	3	0	3	4	3	0	5

Figure 2: Two simple circuits and its corresponding circuit vector ("genetic code")

Modelling the performance of a Unit – The performance of a separation unit operation will depend on a number of factors, including the feed composition and feed rate. We will assume that there are only three components in the feed, namely a waste and two valuable component (in many systems there will actually be a large range of valuable and waste components with different separation performances).

We will assume that the recovery of the material to the valuable output streams follows a first-order kinetic relationship. This means that the recovery of material to the concentrate is proportional to the amount of material in the cell. If the material in the cell is well mixed, then the recovery, R, of material i to the concentrate stream obeys the following relationship:

$$R_i^{Cn} = \frac{k_i^n \tau}{1 + k_i^n \tau}.$$

where k_i^n is the rate constant associated with component i in the *n*th concentrate stream and τ is the average residence time for material in the cell. We will assume that rate constants are as given in the appendix. Note that this model is not precisely representative of any particular physical separator but instead does a good job approximating the behaviour of many different types of unit.

The residence time is the volume of the cell divided by the combined volumetric feed rate of solids and water into the cell. For now we will assume that both solids have a density, ρ, of 3000 kg/m^3 (although in the real world, density differences are important in many kinds of separator) and that the total solids (palusznium+ gormanium + waste) content of the feed by volume, φ, is 10% in all cells. We will also assume that all the cells have a volume $V = 10$ m^3. In this case,

$$\tau = \varphi \frac{V}{\sum_i \frac{F_i}{\rho}}$$

where F_i is the mass flow rate of solid component i in the feed to the cell.

Once the recovery has been calculated, it can be used to calculate the mass flow rates of both components in the tails and concentrate streams from the cell.

Type A: $C_i^1 = F_i \, R_i^{C1}$

$$T_i = F_i - C_i^1 = F_i(1 - R_i^{C1})$$

Type B: $C_i^n = F_i \, R_i^{Cn}$

$$T_i = F_i - C_i^1 - C_i^2 = F_i(1 - R_i^{C1} - R_i^{C2})$$

Where C_i^n is the mass flowrate of material i to the *n*th concentrate output stream and T_i is the mass flowrate of material i to the tails.

As an example calculation, using two materials and two outputs, if $F_w = 90\ kg/s$ (i.e. the waste feed) and $F_G = 10\ kg/s$ (i.e. the gormanium feed) then $\tau = 0.1\ \frac{10}{\frac{90+10}{3000}} = 30\ s$. The recoveries are $R_1 = \frac{30 \cdot 0.006}{30 \cdot 0.006 + 1} \approx 0.152$, $R_G = \frac{30 \cdot 0.0005}{30 \cdot 0.0005 + 1} \approx 0.0148$. This means that the concentrate and tails have the following flowrates: $C_G = 10 \cdot 0.152 \approx 1.52\ kg/s$, $C_w = 90 \cdot 0.0148 \approx 1.33\ kg/s$, Taking the remainders gives the tailings, $T_G \approx 8.48\ kg/s\ T_W \approx 88.7\ kg/s$.

There is one note of warning: In some invalid circuits the feed rate in some cells will iterate towards zero, which will cause issues in the calculation of the residence time. To stop overflow errors in the calculation, set a minimum flow-rate/maximum residence time to stop these errors (a minimum flow-rate of 1×10^{-10} m^3/s would be appropriate).

Finally operating costs are modelled through a penalty function based on a power law applied to the unit volume. See the appendix for the formula to apply. Initially this will just reduce the profit of the circuit. Once we allow circuit volumes to change, it will affect the optimal solution.

Modelling the Circuit – To combine the behaviour of each of the individual units into an overall circuit simulation we must calculate the mass flow rate of each component in each stream. To do this we will assume that the circuit is at steady state, which implies that the flows in each stream do not change with time and that there is no accumulation (i.e., the total feed into a unit is equal to the sum of the flow out of the unit through the two output streams).

As these circuits can (and will typically) involve recycles—that is, one or more of the output streams can feed back into an earlier unit in the circuit—the circuit performance will need to be solved iteratively. As successive substitution of the component mass flows in the streams is likely to converge in these types of problems (this can be proved for linear models, you can look up the proof if you wish) we recommend this approach to start with. It is possible to achieve quicker convergence using a more complex convergence algorithm, though you do still need to ensure stability of convergence.

The following is a simple successive substitution algorithm that is guaranteed to converge *if a valid solution exists*:

1) Give an initial guess for the feed rate of both components to every cell in the circuit
2) For each unit use the current guess of the feed flow-rates to calculate the flow-rate of each component in the concentrate and tailings streams
3) Store the current value of the feed to each cell as an old feed value and set the current value for all components to zero
4) Set the feed to the cell receiving the circuit feed equal to the flow-rates of the circuit feed
5) Go over each unit and add the concentrate and tailings flows to the appropriate unit's feed based on the linkages in the circuit vector. This will also result in an updated estimate for the overall circuit concentrate and tailings streams' flows.
6) Check the difference between the newly calculated feed rate and the old feed rate for each cell. If any of them have a relative change that is above a given threshold (1×10^{-6} might be appropriate) then repeat from step 2. You should also leave this loop if a given number of iterations has been exceeded or if there is another indication of lack of convergence as this will generally indicate an invalid circuit configuration (be aware that as you test this on larger circuits the number of iterations required for convergence of valid circuits will increase).
7) Based on the flow-rates of the overall circuit concentrate stream, calculate a performance value for the circuit. If there is no convergence you may wish to use the worst possible performance as the performance value (the flow-rate of waste in the feed times the value of the waste, which is usually a negative number).

Checking Circuit Validity – A key step before running a simulation is to ensure that the circuit is a valid one. Recall that the algorithm described above for evaluating circuit performance will not converge if the circuit is invalid. Lack of convergence is thus a test of circuit validity. However, as this is a computationally intensive way to evaluate circuit validity, we recommend that pre-requisite validity checks are implemented, based on the following considerations.

For the circuit to be valid a few conditions must be met:

- Every unit must be accessible from the feed. I.e., there must be routes that go forward from one unit to the next starting at the feed and ending at each of the units in the circuit
- Every unit must have a route forward to at least two of the outlet streams. A circuit with no route to any of the outlet streams will result in accumulation and therefore no valid steady state mass balance. If there is a route to only one outlet then the circuit will be able to converge, but there will be one or more units that are not contributing to the separation and could therefore be replaced with a pipe.
- There should be no self-recycle. In other words, no unit should have itself as the destination for any of the three product streams.
- The destinations for the products from a unit should not all be the same unit (i.e. again, we shouldn't be able to replace a separation unit by a pipe)

Note that this is not an exhaustive list of how circuits can be invalid or obviously sub-optimal. You should think about other ways in which circuits can be invalid and do tests for these. Even once you have implemented validity checking based on the above criteria, you should still check for lack of convergence as there are some pathological circuit configurations that you might not think of in the validity testing. Hence, you should still check if the circuit simulation is diverging as this will indicate an invalid circuit (have a maximum number of iterations allowed in the circuit mass balance convergence). It may be insightful to look at some of the circuits that are identified as invalid through non-convergence and see if you can identify why they are invalid.

When doing these checks, you should note that the circuit takes the form of a directed graph. It is often easiest to write recursive functions to traverse the graph, though you should note that, because recycle is allowed (i.e. a unit's output streams can connect to units which flow back into it), you do need to ensure that you don't get stuck going around a recycle loop, i.e. to apply a search algorithm. The easiest way to do this is to mark the units that you have already visited. A simple generic function for using recursion to traverse the circuit is outlined in the section below.

Traversing the Circuit using Recursion

Recursive functions are the easiest way to traverse a tree or graph. This short code snippet demonstrates a function which marks every unit which is accessible from a given unit (i.e. every unit that the product from a given unit can potentially reach). It assumes that the data for each individual unit is stored in a class:

```
class Cunit
{
public:
        // index of the unit to which this unit's concentrate stream is connected
        int conc_num;
        // index of the unit to which this unit's tailings stream is connected
        int tails_num;
        // A Boolean that is changed to true if the unit has been seen
        bool mark;
        …other member functions and variables of Cunit
};
```

And it assumes that there is an array or vector of these units:

```
int num_units;
…set a value to num_units
vector<Cunit> units(num_units);
```

The following function is recursive, which means that it calls other instances of itself within the function.

```
Void mark_units(int unit_num)
{
        if (units[unit_num].mark) // Exit if we have seen this unit already
                return;
        units[unit_num].mark = true; // Mark that we have now seen the unit

//If conc_num does not point at a circuit outlet, recursively call the function
        if (units[unit_num].conc_num < num_units)
                mark_units(units[unit_num].conc_num);
        else


//If tails_num does not point at a circuit outlet, recursively call the function
        if (units[unit_num].tails_num < num_units)
                mark_units(units[unit_num].tails_num);
        else
                …Potentially do something to indicate that you have seen an exit

}
```

To use this function in the code you need to use the specification vector to set the `conc_num`, `inter_num` and `tails_num` values for every unit in the units array:

```
//Set all the cells to unseen
for (int i=0; i<num_units; i++)
        units[i].mark = false;

//Mark every cell that start_unit can see
mark_units(start_unit);

for (int i=0;i<num_units;i++)
        if (units[i].mark)
                …You have seen unit i
        else
                …You have not seen unit i
```

Base Case Circuit Specification

Your challenge is to determine the optimum circuit configuration and performance targeting a circuit with fixed unit type that contains 10 units (7 of type A, 3 of type B) and a circuit of 8 units where you are free to pick the units. It should solve for a total circuit feed of 8 kg/s of palusznium, 12 kg/s of gormanium the valuable materials, and 80 kg/s of the waste material. You will be paid for the palusznium and gormanium streams, and pay operating costs for units according to the schedule in the Appendix.

Your overall task is to provide advice on optimum circuit configuration. Therefore, additional credit will be given to teams that demonstrate a deeper understanding of how optimum circuit configuration depends on the number and type of units, as well as the economic factors through application of their tool. This might include identification of common configuration patterns and how these change with different economic constraints.

What is required of each team?

Software development

Your software tool should comprise four parts, which can be developed independently and should be written in a modular fashion.

1) Create software capable of using a genetic algorithm to optimise a system represented by a specification vector where the performance is based on the evaluation of a fitness function that takes in the sequence of integers, and returns a single floating point number to be maximised.
2) Write a mass balance simulator which can take in a specification vector for the circuit and calculate the mass flows of all components in every stream and ultimately returns a single number to represent the monetary value of the final concentrate output.
3) Write a fast validity checking function that takes in the circuit vector and returns true or false based on its assessment of the circuit validity.
4) Write a post-processing code that can convert any circuit vector into a visualisation (i.e. an image) of the circuit that this vector represents. There are Python and C++ libraries that can assist in this task (note that the circuit is a directed graph). You could also expand the code to able to indicate additional useful information about the circuit.

By developing the software in this modular fashion, it should be possible to use your genetic algorithms on a different problem, to apply a different optimisation algorithm to your circuit simulator, or to change the physical properties of the model. This means that information should be kept with-in the library it's relevant to, but might mean adding additional optional inputs. Your code must include the interfaces specified in the RequiredFunctions.h header file and interface with the supplied ESEGraph library.

Model analysis and performance enhancement

When you have a working tool, you should attempt to use it to address the following:

5) Obtain the optimum circuit configuration for the base case specifications. How quickly and how reliably does the algorithm converge on the optimum and how does this change with the genetic algorithm parameters such as the number of child vectors used and the mutation rate? Note that the reliability with which the algorithm finds the global solution

will often be in conflict with the number of iterations required to find the optimum. Remember to run the code a number of times to see if an improved optimum solution can be found (the more cells in the circuit the more likely that the code will find a local rather than the global optimum).

6) Investigate how the optimum circuit changes as the various problem-specification parameters change, including the number of units, the prices paid for gormanium & palusznium relative to the cost of disposing of the waste material, and the purity of the input feed. Note that you will often have to make large changes in these parameters to drive significant changes in the optimum circuit configuration. Are there any circuit design heuristics that you might recommend based on the observed trends in the optimum configuration? Note that different economic parameters will require different balances between the recovery and purity of the product in the optimum circuit.
7) The fact that the genetic algorithm requires the evaluation of a large number of independent circuit configurations at each iteration means that it is readily parallelised. It is up to you whether you do openMP or MPI based parallelisation (openMP will be easier to implement, but your code will be restricted to working only on a single node). You could also write a script to test different parameter settings by running different instances of your serial code in parallel, though this would get less credit than the implementation of a truly parallel code.

Note that you are not expected to complete all of these tasks, though you should complete tasks 1-5 as a minimum. These tasks also carry the greatest weighting in terms of the marking scheme.

Recommendations for how to proceed to start with

There are four aspects to this project that can be worked on independently before they are brought together in the final code. Before any of these tasks can be attempted, though, a data structure for the circuit vector specification needs to be agreed. Additionally, for testing of the circuit validity and carrying out circuit simulation it would be useful to agree data structures for the individual units, streams etc.

The following are four overall tasks that can be independently tackled:

A. ***Circuit Simulation*** **–** For a given circuit vector you need to be able to calculate the mass balance over all the units and thereby the composition of both of the outlet streams as this is required to produce a single fitness value. You can develop this simulator without validity checks or the genetic algorithm by manually creating a few test vectors. This can be done by drawing a circuit that you can see is valid and then writing out the corresponding circuit vector. In drawing the test circuits, you should design complex circuits with lots of recycles rather than circuits you think may be efficient in order to have a good test of the circuit simulator's convergence behaviour.

You are expected to expose the following interface:

```
double circuit_performance(std::span<const int> const circuit_span);
```

B. ***Circuit Validity*** **–** Having valid circuits is important for convergence of the mass balance. While you could use non-convergence of the mass balance as an indicator of an invalid

circuit, it is much better to explicitly check circuit validity first. Checking circuit validity can be done without the ability to calculate the mass balance and can therefore be done as an independent task (how these circuits might be invalid is discussed in an earlier section).

The following signature of function definition must be used for this function.

```
bool check_validity(std::span<const int> const circuit_span);
```

Alternatively, you could agree an internal circuit representation akin to that used in the section on traversing the circuit and have a shared function which converts the circuit vector into this format for use in both the circuit evaluation and validity checking.

C. ***The Genetic Algorithm*** **–** This can be developed without the circuit simulator as you simply need a fitness function that gives back a value based on a given number vector. While this will ultimately be the value of the concentrate stream as calculated in the circuit simulator, in order to develop the algorithm a simple test case is to choose a random vector as the answer and have the fitness function return a number that represents the difference between this answer vector and test vector. The advantage of this approach is that you know if the answer returned is the correct one. Don't rely on this test problem to tune for the correct parameters to use in the genetic algorithm as this is a dramatically simpler optimisation than the actual one and which should converge much more quickly.

When developing the genetic algorithm (i.e., before the other sub-groups have completed their tasks) you might want to use the following test versions of the above functions (note that the optimum settings for the genetic algorithm will be very different for this test problem compared to the actual problem):

```
bool check_validity(std::span<const int> const circuit_span){

        return true;
}

double circuit_performance(std::span<const int> const circuit_span){
        double Performance =0.0;
        for (int i=0;i<vector_size;i++)
        {
                //answer_vector is a predetermined answer vector (same size as
circuit_span)
                Performance+=(20-abs(circuit_span[i]-answer_vector[i])*100.0;
        }
        return Performance;
}
```

Post Processing **–** The output from the simulator will be a vector of circuit connections and a performance score. This vector can be represented as a directed graph of the nodes in the circuit:

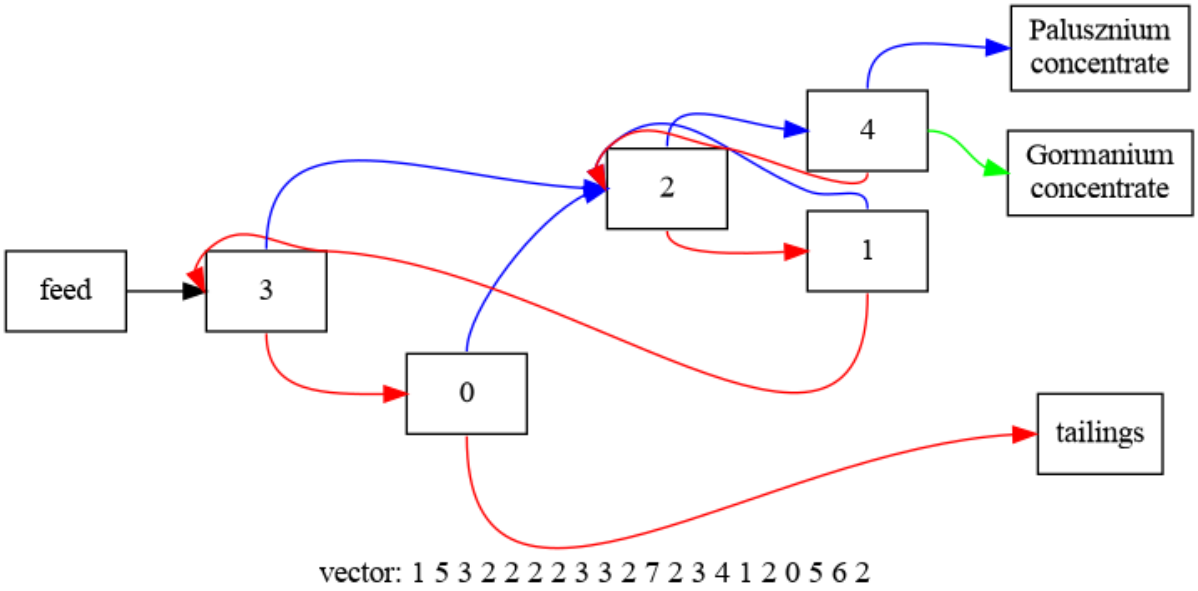


Figure 3 A 5 unit circuit with 4 type A units and 1 type B unit. With the base economic & separation parameters, this circuit returns £320.69 per second, with a 24.0.% Palusnium recovery at 86.7% grade and a 17.3.% Gormanium recovery at 80.4.% grade. This arrangement is likely optimal for this number of units.

Your group can choose to use Python or C++ in this postprocessing task and there are various libraries that can assist with this task. One that we recommend is graphviz (https://graphviz.org) and, in particular, its Digraph functionality. You are free to develop your own tools as needed, using *matplotlib* or a similar library in Python, or any widely distributed C++ library.

Assessment

Your group project will be assessed in four ways:

Software (70 marks)

Software will be assessed based on functionality (50/70 marks) and sustainability (20/70 marks).

Functionality and performance (50 marks): Your software will be primarily assessed on its ability to perform the required tasks:

- A maximum of 10 marks will be awarded to the performance of the genetic algorithm implementation for circuit optimization with fixed and swapable units.
- A maximum of 8 marks will be awarded for the mass balance solver.
- A maximum of 6 marks will be awarded for the circuit validity checking
- A maximum of 6 marks will be awarded for the post-processing routines
- A further 12 marks will be assessed based on additional features and optimisations beyond the direct implementation of the algorithms specified in the description. This could include, but is not limited to, things like parallelisation, modifications and improvements to the algorithms used or improvements in the modularity and reusability of the code.
- 3 marks will be awarded for interfacing with the supplied ESEGraph library
- The final 5 marks will be awarded for the stylistic consistency of your codebase.

Part of the assessment for these marks will be based on the final speed and efficiency of the code, but proper documentation of these additional features is also important so that they are not missed during our assessment. Bonus marks may be awarded for (sensible) originality in any section, so keep your Team's plans secret.

Marks will be deducted for (a) inaccuracy in the solution; (b) bugs or mistakes in implementation; (c) avoidable computational inefficiency, except where it would significantly hinder code reuse or readability. There is no requirement to provide a GUI or other direct interface, but methods to deal with configuration are highly appropriate. If you do provide an interface, there should be a way to easily run your code on the default problem sets via a single command line call.

Sustainability (20 marks): As with all software projects, you should employ all the elements of best practice in software development that you have learned so far. A GitHub repository will be created for your project to host your software. The quality and sustainability of your software and its documentation will be assessed based on your final repository and how it evolves during the week. Please refer to the module handbook for more information about the assessment of software quality. You should include (and extend) proper test cases for the different aspects of the code, including the genetic algorithm, the circuit simulator and the circuit validity checking. Other important aspect of the sustainability of the project include sufficient and clear documentation of the code, clear descriptions of the code's usage, as well as examples .

Presentation (20 marks)

Your project will also be assessed on the basis of a 15-minute video presentation that you must upload to your groups private Teams channel before the deadline of Friday 22nd May, 4:00 pm.

You can record (or edit) the presentation in any software that you like, but if you have no inclinations of your own, it's fine to record your presentation as Teams meeting (including appropriate visual aids).

You should view the presentation as your report to the project client. They have tasked you with determining the optimum circuit configuration, with the specified problem constraints. To be awarded the contract for further work, they will want to know how you arrived at your answer and see evidence that your tool can be trusted and performs well on average. However, they will also be interested in any insight you have gleaned into the sensitivity of the optimum circuit configuration to changes to the problem constraints, particularly the economic factors.

Your presentation should provide the following information:

- A discussion of the key elements of the solution method and, in particular, any improvements and optimisations made beyond the algorithms presented here.
- Your solution for the fixed base case with 10 units based on the specified physical process parameters and economic variables.
- A brief discussion of the continuous case, including an assessment of the capability of your algorithm.
- An investigation into your program's performance, including both speed of convergence and the robustness of the final solution to the initial population of vectors, and how this is influenced by the genetic algorithm's parameters.
- An investigation into how and why the optimum circuit configuration changes with the input variables, in particular the economic factors.

Teamwork (peer assessment; 10 marks)

Following the completion of the we, you will be tasked to complete a self-evaluation of your group's performance, as with previous group projects. This will inform the teamwork component of your mark. Please refer to the ACS guidelines for more information about the assessment of teamwork.

Technical requirements

You should use the assigned GitHub repository exclusively for your project

Your software should be predominantly written in C++, although you can write post-processing modules in Python if you wish (and link the two languages in a single executable if you choose)

Your program should be written in the C++20 ANSI standard C or above, and be written to compile on multiple operating systems.

Your core program should be capable of being run from the command line without any additional interactive user input, so that it could be submitted to run on an HPC job system (it may take non-interactive inputs from the command line or from additional configuration files). Feel free to include example submission scripts in your submission.

You should use GitHub Actions for any automated testing that you implement, but are totally free to modify or replace the existing testing workflow in the template repository you are supplied. Please **do not use Mac runners** on GitHub unless you have discussed the reasons you believe linux runners are inappropriate with us first.

Appendix: Physical parameters for the base cases

Number of units :

Fixed units case: 10

Swapable units case: 8

Number of unit outputs:

For type A units: 2 (concentrate & waste)

For type B units: 3 (2 concentrate, 1 waste

Unit types:

Fixed case: 7 (type A)_ and 3 (type B_

Swapable case: *to be optimized by your code*

Feed (one input):

Palusznium	Gormanium	Waste
8 kg/s	12 kg/s	80 kg/s

Separation constants:

For type A units:

	Palusznium	Gormanium	Waste
k_i^1 value	0.008 s^{-1}	0.006 s^{-1}	0.0005 s^{-1}

For type B units:

	Palusznium	Gormanium	Waste
k_i^1 value	0.007 s^{-1}	0.001 s^{-1}	0.001 s^{-1}
k_i^2 value	0.001 s^{-1}	0.006 s^{-1}	0.001 s^{-1}

Unit volume: 10 m^3

Value of product stream based on content (two output streams)

Product stream	Palusznium	Gormanium	Waste
In Palusznium Stream	+£120 per kg	-£20 per kg	-£500 per kg
In Gormanium Stream	-£5 per kg	+£100 per kg	-£500 per kg

Circuit operating cost per second (penalty):

For type A units: £10.00 per second

For type B units: £12.00 per second